ACTIVITY NO. 1

REVIEW OF C++ PROGRAMMING	
Course Code: CPE010	**Program:** Computer Engineering
Course Title: Data Structures and Algorithms	**Date Performed: July 10-14, 2026**
Section: CPE21S2	**Date Submitted: July 14, 2026**
Name: Jaime Luis M. Demain	**Instructor: Engr. Jimlord M. Quejado**

1. Objective(s)

- Implement basic programming and OOP in C++

2. Intended Learning Outcomes (ILOs)

After this module, the student should be able to:

a. Create code that follows the basic C++ code structure;
b. Implement appropriate class definition and instances based on given requirements;
c. Solve different problems using the C++ programming language.

3. Discussion

Part A: Introduction to C++ Code

Structure of C++ Code

Sections	**Sample Code**
Header File Declaration Section	`#include<iostream>` `using namespace std;`
Global Declaration Section	`int count = 0;`

Class Declaration and Method Definition Section	`class`
	`rectangle{ private:`
	`  double recLength, recWidth;`
	
	`public:`
	`  rectangle(double L, double W);`
	`  void setLength(double L);`
	`  void setWidth(double W);`
	`  double getPerimeter();`
	`};`

Main Function	`int main(){`
	`  rectangle shape1(2, 5);`
	`  std::cout << "The perimeter of the rectangle is " << shape1.getPerimeter() << ".\n";`
	`  std::cout << count << " number of objects created.";`
	`  return 0;`
	`}`

Method Definition	`rectangle::rectangle(double L, double W)`
	`  { recLength = L;`
	`  recWidth = W;`
	`  count++;`
	`}`

```
void rectangle::setLength(double L)
    { recLength = L;
}

void rectangle::setWidth(double W)
    { recWidth = W;
}

double rectangle::getPerimeter()
    { return (2*recLength) +
    (2*recWidth);
}
```

It is not required for all sections to have code for every use-case. However, for best practices you would prefer to have an overall structure to follow to increase code readability and reusability.

Data Types

d. Primary Data Type: int, float, char and void
e. User defined data type: structure, union, class, enumeration
f. Derived data type: array, function, pointer, reference

Local & Global Variables

```
#include <iostream>
using namespace std;

int globalVal = 0; //Global Variable

int main(){
    int localVal = 5; //Local Variable

    std::cout << "Global Variable has value " << globalVal << ".\n";
    std::cout << "Local Variable has value " << localVal << ".\n";

    return 0;
}
```

Operators

Arithmetic	Relational	Logical
Addition +	Greater than >	AND &&
Subtraction –	Less than <	OR ||
Multiplication *	Greater than or equal >=	NOT !
Division /	Less than or equal <=	
Modulo %	Equal ==	
Increment ++	Not equal !=	
Decrement --		

Bitwise Operators

Let A = 60 and B = 13. Binary values are as follows:

```
A = 0011 1100
B = 0000 1101
```

Bitwise AND -> &	`A & B`	`0000 1100`
Bitwise OR -> |	`A | B`	`0011 1101`
Bitwise XOR -> ^	`A ^ B`	`0011 0001`
Bitwise Complement -> ~	`~A`	`1100 0011`

Assignment Operator

Assign a value to a variable. Example:

Assign the value 20 to a variable A.

```
int A = 20;
```

The assignment operator is a basic component denoted as "=".

Part B: Classes and Objects using C++

To create a class use the class keyword. Syntax is:

```
class myClass
  { public:
      int myNum;
      string myString;
};
```

`public` here is an access specifier. It indicates that the attributes and methods listed under it are accessible outside the class. A simple table is provided below to summarize the access specifiers used in c++.

Specifiers	Within same class	In derived class	Outside the class
`private`	Yes	No	No
`protected`	Yes	Yes	No
`public`	Yes	Yes	Yes

We can then create an object from this class:

```
int main(){
      //this creates the object
      myClass object1;

      //this accesses the public attributes
      object1.myNum = 5;
      object1.myString = "Sample";

      return 0;
}
```

4. Materials and Equipment

Personal Computer with C++ IDE
Recommended IDE:

- CLion (must use TIP email to download)
- DevC++ (use the embarcadero fork or configure to C++17)

5. Procedure

ILO A: Create Code That Follows the Basic C++ Code Structure

For this activity, you have to demonstrate the use of a **function prototype**. The section on class declaration and method definition will be used for the function prototype and the function will be defined in the follow method definition section after the main function.

A function prototype in c++ is a declaration of the name, parameters and return type of the function before its definition. Write a C++ code the satisfies the following:

- Create a function that will take two numbers and display the sum.
- Create a function that will return whether variable A is greater than variable B.
- Create a function that will take two Boolean values and display the result of all logical operations then return true if it was a success.

Note:

- The driver program must call each function.
- The definitions must be after the main function.

ILO B: Implement Appropriate Class Definition and Instances Based on Given Requirements

In this section, the initial implementation for a class **triangle** will be implemented. The step-by-step procedure is shown below:

Step 1. Include the necessary header files. For this one, we only need `#include <iostream>`

Step 2. Create the triangle class. Assign it with private variables: totalAngle, angleA, angleB, and angleC.

```
class
Triangle{ priva
te:
    double totalAngle, angleA, angleB, angleC;
```

Step 3. We then create public methods. The constructor must allow for creation of the object with 3 initial angles to be stored in our previously defined variables `angleA, angleB` and `angleC`. Another method has to be made if the user wants to change the initial values, this will also accept 3 arguments to change the values in `angleA, angleB` and `angleC`. Lastly, a function to validate whether the given values make our shape an actual triangle.

```
public:
    Triangle(double A, double B, double C);
    void setAngles(double A, double B, double C);
    const bool validateTriangle();
};
```

Step 4. Define the methods.

```
Triangle::Triangle(double A, double B, double C)
    { angleA = A;
```

```
        angleB = B;
        angleC = C;
        totalAngle = A+B+C;
    }

    void Triangle::setAngles(double A, double B, double C)
        { angleA = A;
        angleB = B;
        angleC = C;
        totalAngle = A+B+C;
    }

    const bool Triangle::validateTriangle()
        { return (totalAngle <= 180);
    }
```

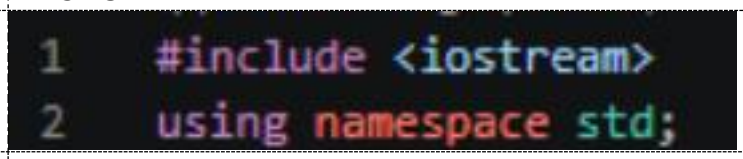
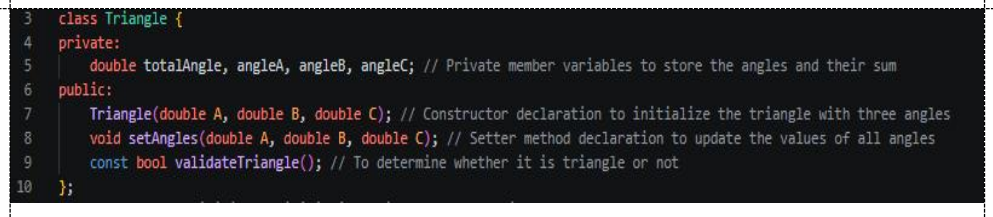
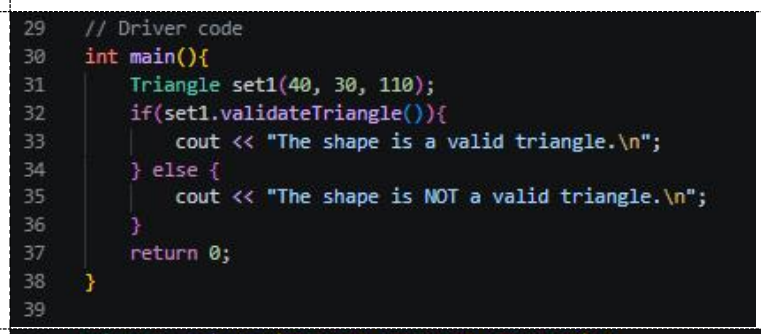
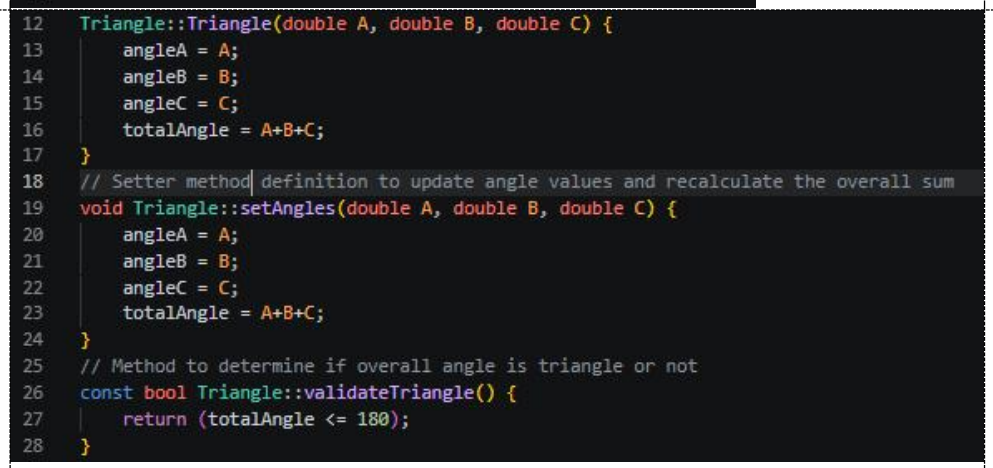
Step 5. Create the driver code.

```
    int main(){
        //driver code
        Triangle set1(40, 30, 110);
        if(set1.validateTriangle()){
            std::cout << "The shape is a valid triangle.\n";
        } else {
            std::cout << "The shape is NOT a valid triangle.\n";
        }

        return 0;
    }
```

Include the output of running this code in section 6. Note your observations and comments.

6. Output

Sections	Answer
Header File Declaration Section	
Global Declaration Section	N/A
Class Declaration and Method Definition Section	
Main Function	
Method Definition	

```
#include <iostream>
using namespace std;
```

```
class Triangle {
private:
    double totalAngle, angleA, angleB, angleC; // Private member variables to store the angles and their sum
public:
    Triangle(double A, double B, double C); // Constructor declaration to initialize the triangle with three angles
    void setAngles(double A, double B, double C); // Setter method declaration to update the values of all angles
    const bool validateTriangle(); // To determine whether it is triangle or not
};
```

```
// Driver code
int main(){
    Triangle set1(40, 30, 110);
    if(set1.validateTriangle()){
        cout << "The shape is a valid triangle.\n";
    } else {
        cout << "The shape is NOT a valid triangle.\n";
    }
    return 0;
}
```

```
Triangle::Triangle(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A+B+C;
}
// Setter method definition to update angle values and recalculate the overall sum
void Triangle::setAngles(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A+B+C;
}
// Method to determine if overall angle is triangle or not
const bool Triangle::validateTriangle() {
    return (totalAngle <= 180);
}
```

Table 1-1. C++ Structure Code for Answer

ILOB.cpp > setAngles(double, double, double)

```cpp
#include <iostream>
class Triangle {
private:
    double totalAngle, angleA, angleB, angleC; // Private member variables to store the angles and their sum
public:
    Triangle(double A, double B, double C); // Constructor declaration to initialize the triangle with three angles
    void setAngles(double A, double B, double C); // Setter method declaration to update the values of all angles
    const bool validateTriangle(); // To determine whether it is triangle or not
};
// Constructor definition to initialize private member variables and calculate total angle
Triangle::Triangle(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A+B+C;
}
// Setter method definition to update angle values and recalculate the overall sum
void Triangle::setAngles(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A+B+C;
}
// Method to determine if overall angle is triangle or not
const bool Triangle::validateTriangle() {
    return (totalAngle <= 180);
}
// Driver code
int main(){
    Triangle set1(40, 30, 110);
    if(set1.validateTriangle()){
        std::cout << "The shape is a valid triangle.\n";
    } else {
        std::cout << "The shape is NOT a valid triangle.\n";
    }
    return 0;
}

```

Table 1-2. ILO B output observations and comments.

```cpp
#include<iostream>
#include<iomanip>
using namespace std;


int sumOftwo (int num1, int num2){
    int sum = num1 + num2;
    return sum;
}
void greaterOftwo (int varA, int varB){
    if (varA > varB) {
        cout << varA << " is greater than " << varB << endl;
        }
    else {
        cout << varA << " is less than " << varB << endl;
        }
}
bool logicalOp (int x ,int y){
    cout << "Logical AND: " << (x && y) << endl;
    cout << "Logical OR: " << (x || y) << endl;
    cout << "Logical XOR: " << (x != y) << endl;
    cout << "Logical NOT x: " << (!x) << endl;
    cout << "Logical NOT y: " << (!y) << endl;
    return true;
}


int main(){
    //Create a function that will take two numbers and display the sum.
    int sumOftwo (int num1, int num2);
    cout << "The sum of two number is " << sumOftwo(7,11) << endl;
    //Create a function that will return whether variable A is greater than variable B.
    greaterOftwo(7,11);
    //Create a function that will take two Boolean values and display the result of all logical operations then return true if it was a success.
    bool success = logicalOp(true, true);
    return 0;
}

```

Table 1-3. Output of ILO A

7. Supplementary Activity

ILO C: Solve Different Problems using the C++ Programming Language

The supplementary activities are meant to gauge your ability in using C++. The problems below range from easy to intermediate to advanced problems. Note your difficulties after answering the problems below.

1. Create a C++ program to swap the two numbers in different variables.
2. Create a C++ program that has a function to convert temperature in Kelvin to Fahrenheit.
3. Create a C++ program that has a function that will calculate the distance between two points.
4. Modify the code given in ILO B and add the following functions:
    a. A function to compute for the area of a triangle
    b. A function to compute for the perimeter of a triangle
    c. A function that determines whether the triangle is acute-angled, obtuse-angled or 'others.'

```cpp
#include<iostream>
#include <cmath> // Include the cmath library for mathematical functions
using namespace std;

//Function for Kelvin to Fahrenheit conversion
double convertKelvinToFahrenheit(double kelvin) {
    return (kelvin - 273.15) * 9/5 + 32;
}
//Function to find distance between two points in 2D space
double distance(double x1, double y1, double x2, double y2) {
    return sqrt(pow(x2 - x1, 2) + pow(y2 - y1, 2));
}

int main(){
    //Create a C++ program to swap the two numbers in different variables.
    double num1, num2, temp;
    cout << "Enter the first number: ";
    cin >> num1;
    cout << "Enter the second number: ";
    cin >> num2;
    cout << "Before swapping: " << num1 << " and " << num2 << endl;
    //Swapping the two numbers using a temporary variable
    temp = num1;
    num1 = num2;
    num2 = temp;
    cout << "After swapping: " << num1 << " and " << num2 << endl;

    //Create a C++ program that has a function to convert temperature in Kelvin to Fahrenheit.
    double kelvin;
    cout << "Enter temperature in Kelvin: ";
    cin >> kelvin;
    cout << "Temperature in Fahrenheit: " << convertKelvinToFahrenheit(kelvin) << endl;

    //Create a C++ program that has a function that will calculate the distance between two points.
    double x1, y1, x2, y2;
    cout << "Enter coordinates of first point (x1 y1): ";
    cin >> x1 >> y1;
    cout << "Enter coordinates of second point (x2 y2): ";
    cin >> x2 >> y2;
    cout << "Distance between the two points: " << distance(x1, y1, x2, y2) << endl;
    return 0;
}

```

Table 2-1. Item 1-3

```cpp
#include <iostream>
#include <string>
using namespace std;

class Triangle {
private:
    double totalandPerimeterAngle, angleA, angleB, angleC, areaAngle; // Private member variables to store the angles, their sum, and their area
public:
    Triangle(double A, double B, double C); // Constructor declaration to initialize the triangle with three angles
    void setAngles(double A, double B, double C); // Setter method declaration to update the values of all angles
    const bool validateTriangle(); // To determine whether it is triangle or not
};
// Constructor definition to initialize private member variables and calculate total angle
Triangle::Triangle(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
    totalandPerimeterAngle = A+B+C;
    areaAngle = (A+B+C)/3;
}
// Setter method definition to update angle values and recalculate the overall sum
void Triangle::setAngles(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
    totalandPerimeterAngle = A+B+C;
    areaAngle = (A+B+C)/3;
}
// Method to determine if overall angle is triangle or not
const bool Triangle::validateTriangle() {
    return (totalandPerimeterAngle <= 180);
}
//A function that determines whether the triangle is acute-angled, obtuse-angled or 'others.'
string determineTriangleType(double angleA, double angleB, double angleC) { // Function to determine the type of triangle based on its angles
        if (angleA < 90 && angleB < 90 && angleC < 90) {
            return "Acute-angled triangle";
        } else if (angleA > 90 || angleB > 90 || angleC > 90) {
            return "Obtuse-angled triangle";
        } else {
            return "Right-angled triangle";
        }
    }
// Driver code
int main(){
    Triangle set1(40, 30, 110);
    if(set1.validateTriangle()){
        cout << "The shape is a valid triangle.\n";
    } else {
        cout << "The shape is NOT a valid triangle.\n";
    }
    Triangle set2(90, 60, 60);
    string triangleType = determineTriangleType(90, 60, 60);
    cout << "The triangle is: " << triangleType << std::endl;
    return 0;
}
```

Table 2-1. Item 4

8. Conclusion

Provide the following:

- Summary of lessons learned
- Analysis of the procedure
- Analysis of the supplementary activity
- Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?

Answer: The main takeaway from this activity is that I was able to refresh my mind on fundamental C++ concepts like variable swapping, mathematical computations, conditional branching structures, and basic class implementations. While I have worked with Boolean logic and functions before in Python, this was my first time implementing them in C++ to handle logical operations. For the procedural and supplementary tasks, I applied object-oriented programming principles to execute a Triangle class that checks for structural validity, and I expanded that code to calculate triangle areas and identify specific triangle types. The supplementary problems effectively gauged my C++ problem-solving abilities by challenging me to implement a classic "temp" variable swap mechanism alongside standard formulas for Kelvin conversion and coordinate distances. Overall, I feel good about my progress, but the primary areas I still need to work on are mastering complex Boolean function implementations and getting more comfortable executing classes cleanly so I can build more advanced C++ programs moving forward.

9. Assessment Rubric